

Sprawozdanie – Laboratorium 1:

ExpressJS i pierwsze REST API

Przedmiot: Zaawansowane Technologie Programowania Aplikacji Internetowych

Temat: Room Booking API (Podstawy REST API)

Student: Kacper Gumulak – 151872

GitHub: github.com/zephir-x/pk-ztpai

1. Krótki opis rozwiązania

Zadanie polegało na zbudowaniu prostego backendu w architekturze REST API przy użyciu środowiska Node.js oraz frameworka ExpressJS. Aplikacja służy do zarządzania salami (odczyt dostępnych sal) oraz ich rezerwacjami (tworzenie, filtrowanie, anulowanie). Zgodnie z wytycznymi, na tym etapie nie zastosowano trwałej bazy danych – wszelkie dane (stan aplikacji) są przechowywane w pamięci operacyjnej wewnątrz zwykłych tablic języka JavaScript (rooms oraz reservations).

Rozwiązanie demonstruje zrozumienie cyklu Request-Response. Poprawnie zaimplementowano parsowanie danych z adresu URL (parametry dynamiczne req.params oraz zapytania req.query), odczytywanie ciała żądania (req.body jako JSON), jak również walidację danych wejściowych i reguły biznesowe. Aplikacja dba o zwracanie odpowiednich kodów statusu HTTP (200, 201, 204, 400, 404, 409) w zależności od wyniku operacji. Testowanie endpointów przeprowadzono za pomocą wbudowanego w środowisko JetBrains Rider klienta HTTP.

2. Realizacja zadań

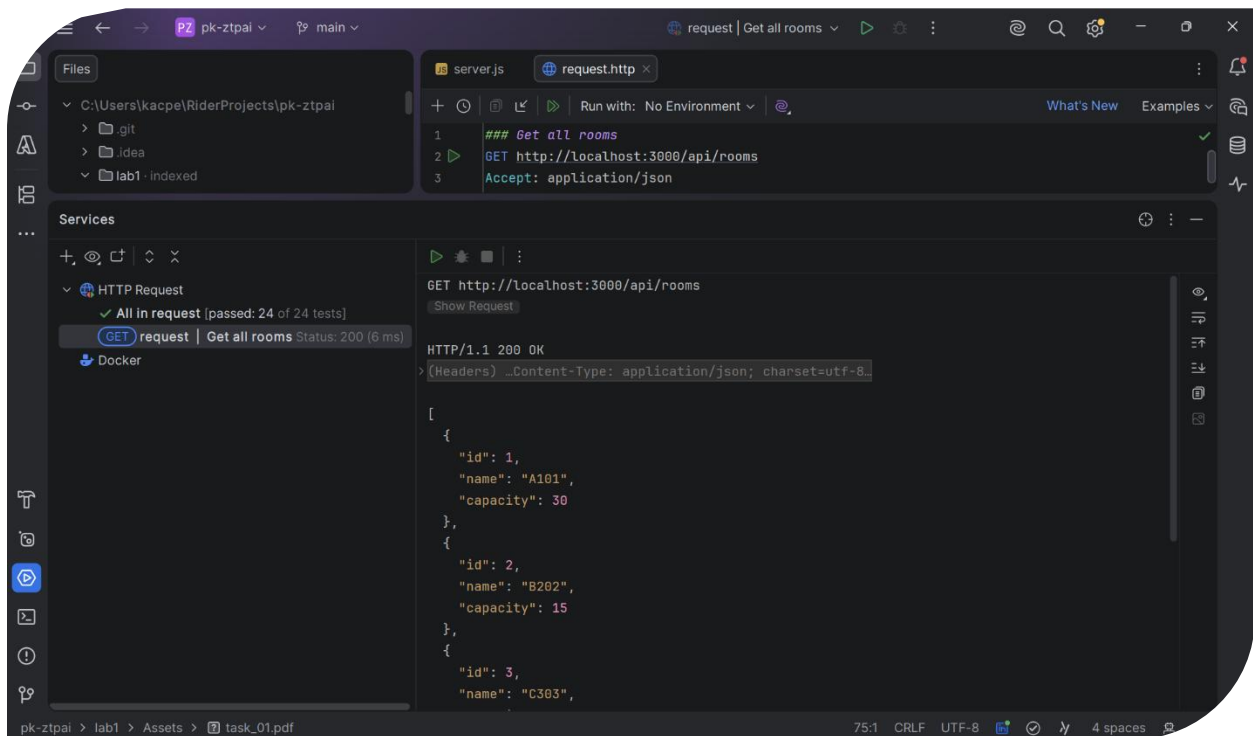
Zadanie 1: Rooms API (Pobieranie zasobów)

Zaimplementowano dwa endpointy typu GET. Pierwszy z nich zwraca całą tablicę dostępnych sal, natomiast drugi pozwala na wyszukanie konkretnej sali na podstawie przekazanego w adresie ID. W przypadku braku sali zwracany jest odpowiedni błąd.

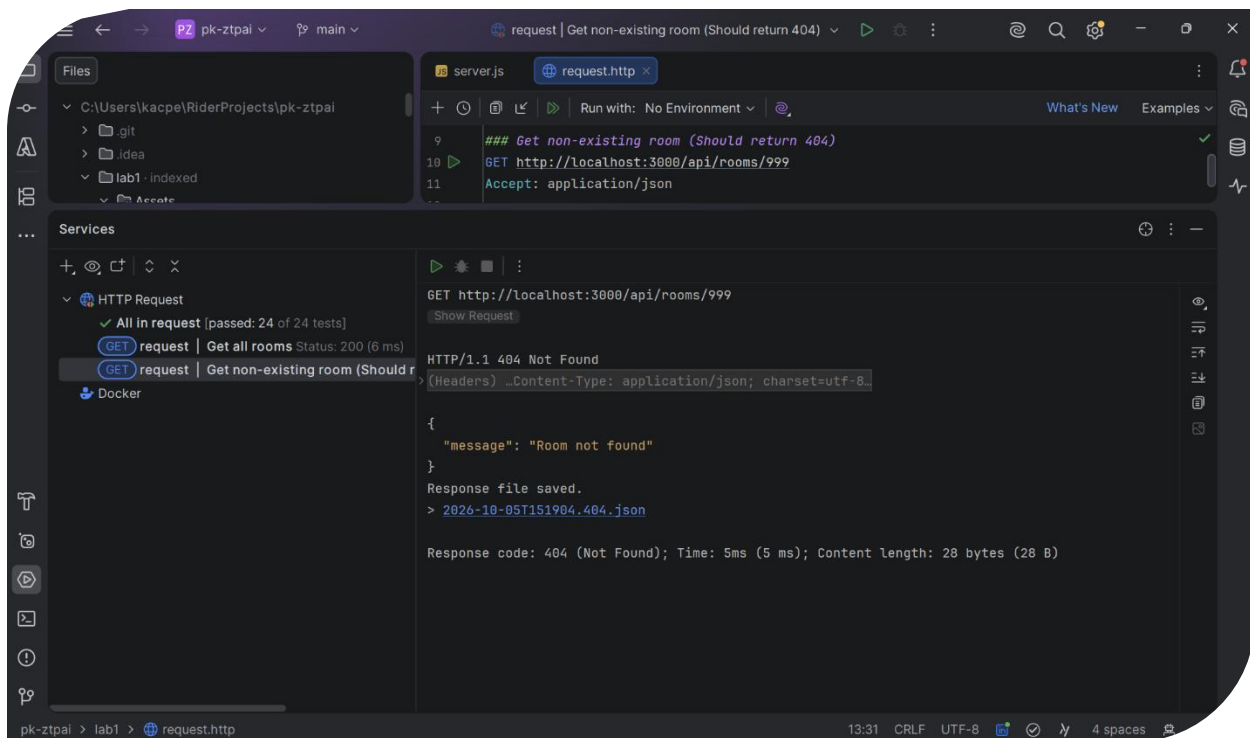
Fragment kodu (wyszukiwanie pojedynczej sali):

```
app.get("/api/rooms/:id", (req, res) => {
  const id = Number(req.params.id);
  const room = rooms.find(r => r.id === id);

  if (!room) {
    return res.status(404).json({ message: "Room not found" });
  }
  res.json(room);
});
```



Zrzut ekranu z Ridera pokazujący poprawny request `GET /api/rooms` (lista wszystkich sal wraz z kodem 200 OK).



Zrzut ekranu z Ridera pokazujący request obsługi błędu: GET /api/rooms/999 (brak sali) zwracający odpowiedź w formacie JSON oraz status 404 Not Found.

Zadanie 2: Tworzenie rezerwacji (Walidacja danych)

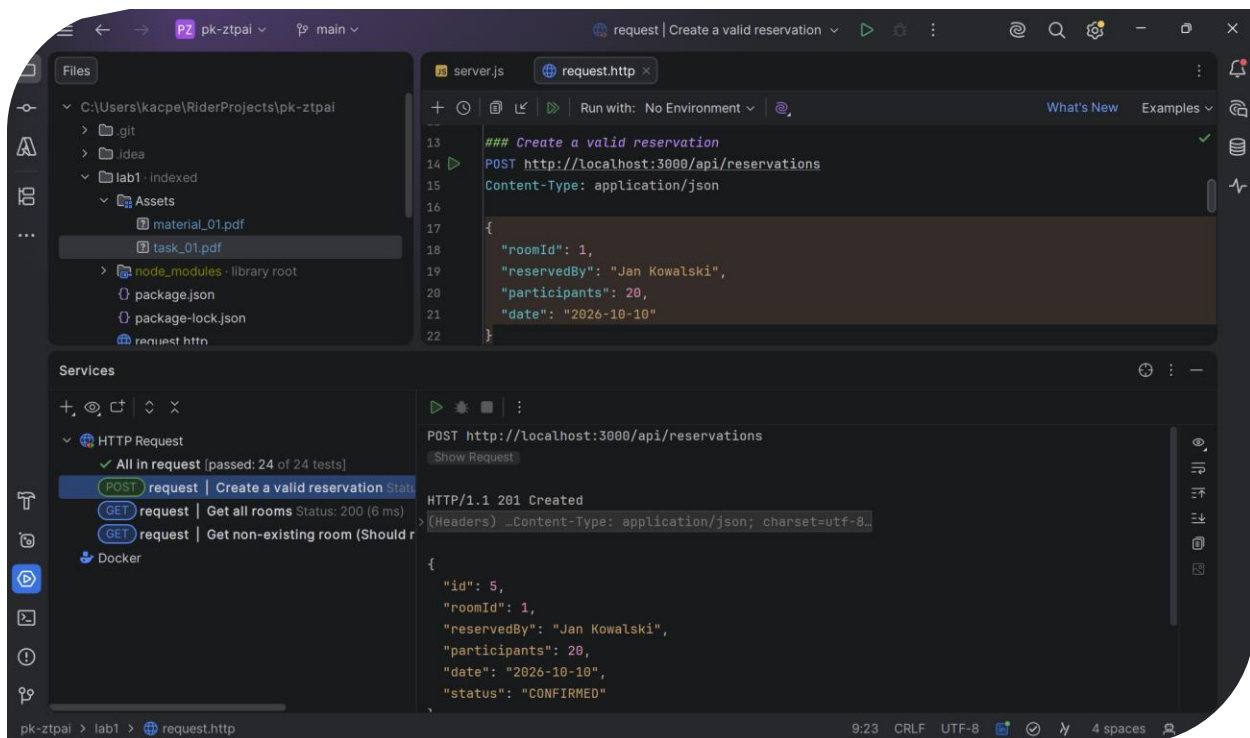
Dodano obsługę żądań POST umożliwiających tworzenie rezerwacji. Zadbano o bezpieczeństwo aplikacji – nie ufając klientowi, system weryfikuje istnienie i poprawność podanych danych, zanim zapisze cokolwiek w pamięci.

Fragment kodu (Walidacja struktury danych):

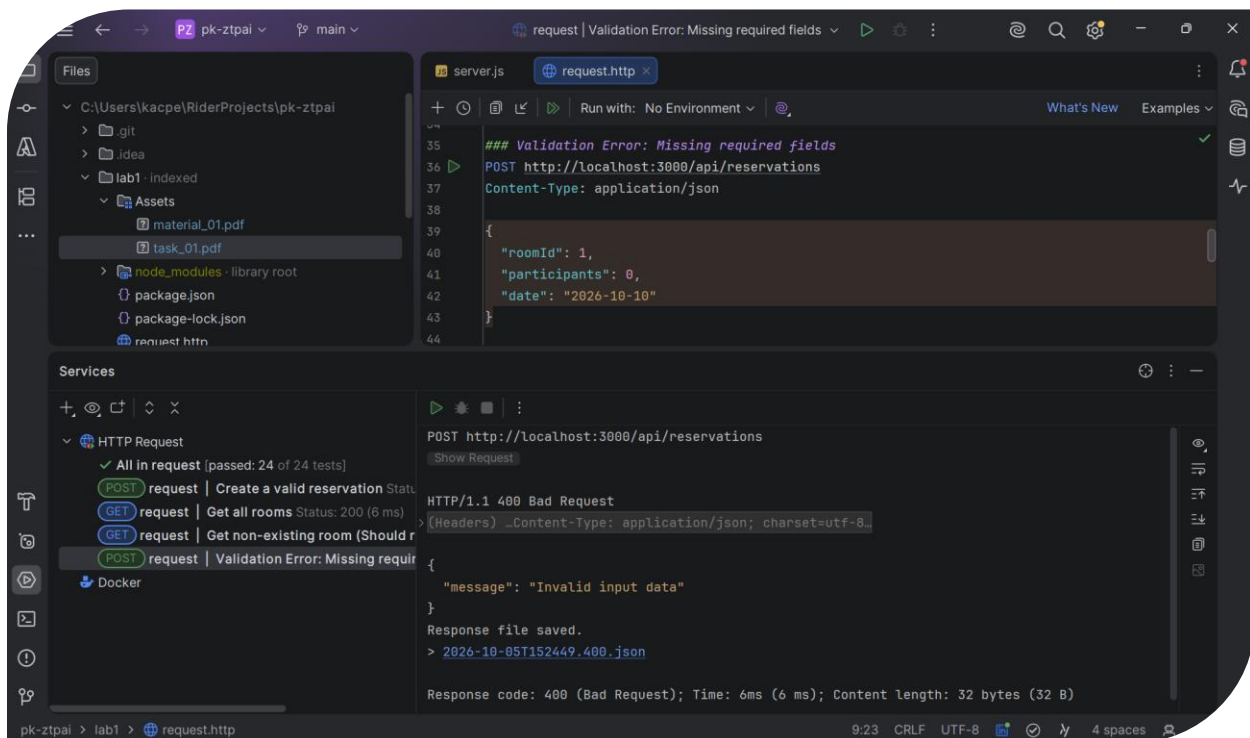
```
app.post("/api/reservations", (req, res) => {
  const { roomId, reservedBy, participants, date } = req.body;

  // Validate incoming data structure and requirements
  if (!roomId || !reservedBy || typeof participants !== 'number' ||
    participants <= 0 || !date) {
    return res.status(400).json({ message: "Invalid input data" });
  }

  // (...) dalsza logika
```



Zrzut ekranu pokazujący poprawny request `POST /api/reservations` (wszystkie dane prawidłowe) oraz odpowiedź ze statusem `201 Created` i polem `status: "CONFIRMED"`.



Zrzut ekranu pokazujący błędny request `POST` np. bez pola `reservedBy`.

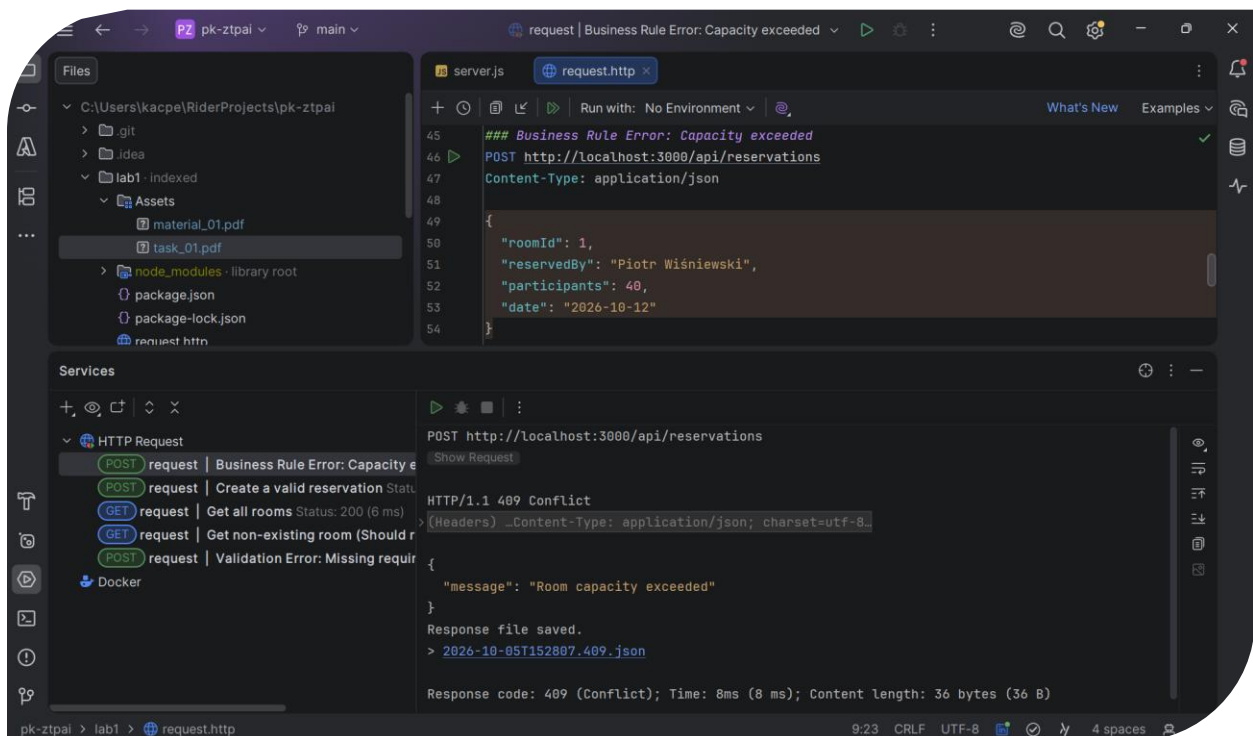
Zadanie 3: Business rule (Pojemność sali)

Logika rezerwacji została rozbudowana o regułę biznesową weryfikującą, czy liczba uczestników nie przekracza dopuszczalnej pojemności danej sali.

Fragment kodu (Reguła biznesowa):

```
// (...) wewnątrz endpointu POST /api/reservations
const room = rooms.find(r => r.id === roomId);

// Business rule: Check if room capacity is exceeded
if (participants > room.capacity) {
  return res.status(409).json({ message: "Room capacity exceeded"
});
}
```



Zrzut ekranu pokazujący próbę utworzenia rezerwacji np. dla 40 osób w sali o pojemności 30.

Zadanie 4: Pobieranie i filtrowanie rezerwacji (Query parameters)

Zbudowano mechanizm pozwalający na pobranie rezerwacji z użyciem GET /api/reservations. Dodano możliwość dynamicznego filtrowania zawartości tablicy, wykorzystując zmienną z paska adresu (tzw. Query Parameter roomId).

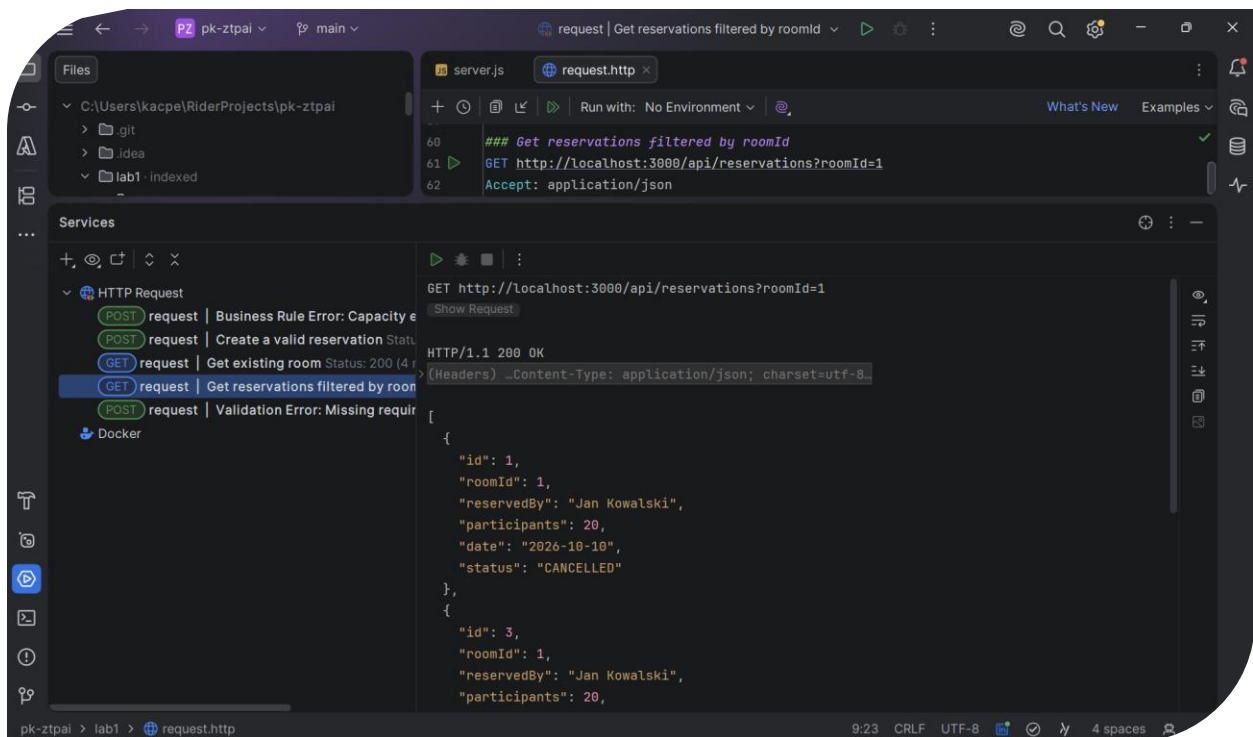
Fragment kodu (Filtrowanie za pomocą req.query):

```
app.get("/api/reservations", (req, res) => {
  const roomId = req.query.roomId;

  if (!roomId) {
    return res.json(reservations);
  }

  const parsedRoomId = Number(roomId);
  const filteredReservations = reservations.filter(r => r.roomId ===
parsedRoomId);

  res.json(filteredReservations);
});
```



Zrzut ekranu pokazujący zapytanie z parametrem: np. GET /api/reservations?roomId=1.

Zadanie 5: Anulowanie rezerwacji

Implementacja endpointu wykorzystującego metodę PATCH do celowej, częściowej modyfikacji istniejącego zasobu. Proces ten nie usuwa dokumentu, a jedynie modyfikuje jego pole status zachowując pełną historię zasobu w zgodzie z dobrymi praktykami.

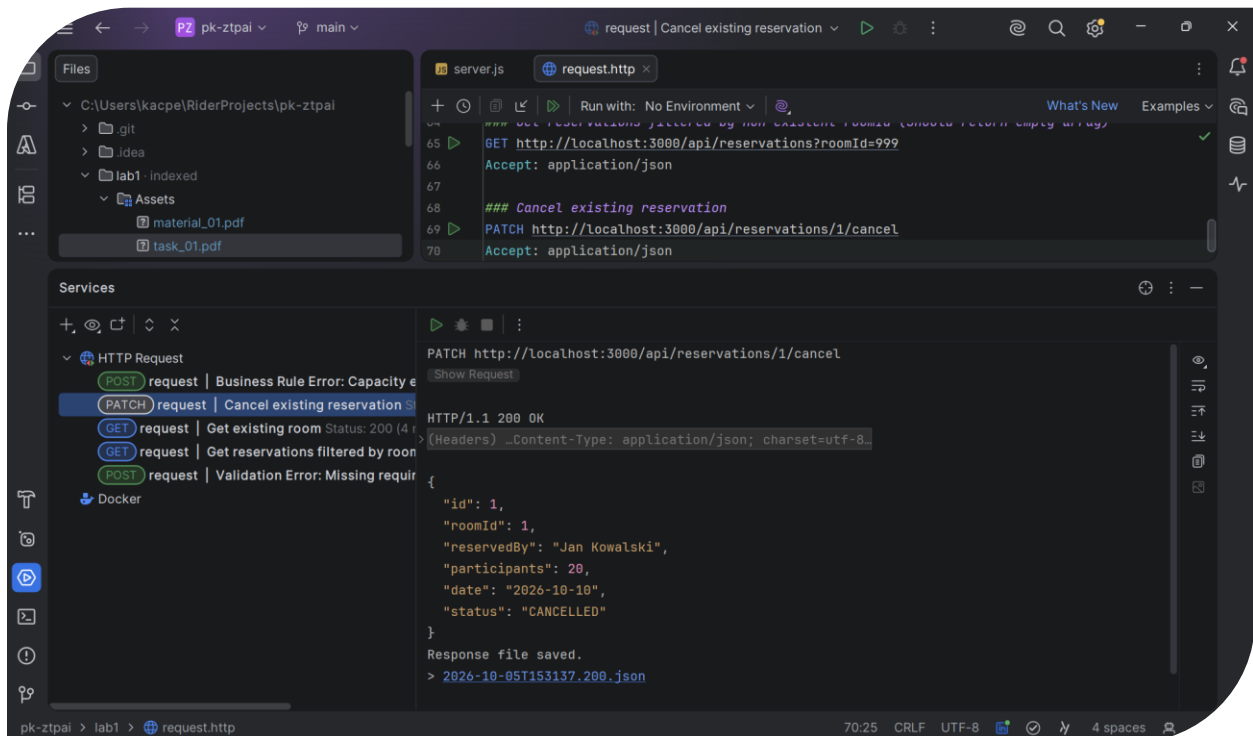
Fragment kodu (Częściowa aktualizacja PATCH):

```
app.patch("/api/reservations/:id/cancel", (req, res) => {
  const id = Number(req.params.id);
  const reservation = reservations.find(r => r.id === id);

  if (!reservation) {
    return res.status(404).json({ message: "Reservation not found"
  });
  }

  if (reservation.status !== "CANCELLED") {
    reservation.status = "CANCELLED";
  }

  res.status(200).json(reservation);
});
```



Zrzut ekranu pokazujący request PATCH /api/reservations/1/cancel.